Modelo de Programacion Funcional

El modelo de programacion funcional es un paradigma de programacion declarativa que basa la construccion de programacion en el uso de funciones matematicas puras, evitando el cambio de estado y los efectos secundarios. En lugar de describir paso a paso como debe ejecutarse un algoritmo, se especifica que se requiere obtener mediante la composicion de funciones que transforman datos

Caracteristicas principales

- Funciones puras: Dada la misma entrada, siempre produce la misma salida y no genera efectos secundarios

- Inmutabilidad: Los datos no se modifican, en su lugar se crean nuevos valores a partir de los existentes

- Funciones como valores de primera clase: Las funciones pueden asignarse a variables, pasarse como argumento y devolverse como resultados

- Recursividad: Al no haber bucles tradicionales basados en mutacion de contadores, la repeticion se logra mediante funciones recursivas

- Ausencia de estado compartido: Cada funcion opera sobre sus propios datos de entrada, reduciendo dependencias ocultas

Lenguajes representativos: Algunos lenguajes diseñados con fuerte orientacion funcional son Haskell, Lisp, Schene Erlang, F# y Scala, Otros como Java Script, Python o Java admiten estilos funcionales aunque no sean puramente funcionales

Elixir

Elixir es un lenguaje de programacion funcional, concurrente y de tipado dinamico se ejecutan en maquina virtual de Erlang (BEAM) y compila a bytecode de Erlang, lo que significa que puede usar todas las librerias de Erlang sin penalizacion de rendimiento

Caracteristicas principales

Concurrencia masiva: usa procesos ligeros que se comunican por paso de mensajes (modelo de actores) capaz de manejar millones de conexiones simultaneas

Tolerancia a fallos: gracias al sistema de supervision de OTP los procesos fallan se reinician automaticamente sin caida del sistema

Sintaxis amigable: inspirada en Ruby a diferencia de la sintaxis de Erlang

Programacion funcional: datos inmutables pattern matching funciones de orden superior

Whatsapp esta escrito principalmente en Erlang y no en Elixir, la relacion es la siguiente

1 WhatsApp escala sobre BEAM: su servidor backend, que llego a manejar mas de 1000 millones de usuarios esta construido en Erlang sobre la misma maquina virtual que usa Elixir

Elixir es Erlang modernizado diseñado precisamente para aprovechar las capacidades de concurrencia y tolerancia a fallos de Erlang/BEAM con una sintaxis mas productiva y familiar (estilo Ruby)

Erlang

Erlang es un lenguaje de programación funcional y concurrente para sistemas de conmutación telefónica: redes que deben mantener millones de llamadas simultáneas y que no pueden caerse nunca. Su nombre viene de Ericsson lenguaje

Características clave

Procesos ligeros: en lugar de hilos del sistema operativo, erlang usa procesos administrados por su máquina virtual BEAM, cada proceso cuesta unos 300 bytes – 2 kB de memoria frente a 1-8 MB de un hilo de Java o C++.

Modelo de actores: los procesos no comparten memoria se comunican únicamente por paso de mensajes

Filosofía let it crash: si un proceso falla un supervisor lo reinicia automáticamente, sin afectar el resto del sistema

Carga código en caliente: permite actualizar el servidor sin desconectar a los usuarios conectados

WhatsApp construyó todo su backend sobre Erlang

Servidores de chat: Cada servidor corre Erlang sobre FreeBSD y mantiene alrededor de 2 millones de conexiones concurrentes cada usuario conectado tiene su propio proceso Erlang que maneja su sesión y estado

Enrutamiento de mensajes, Tolerancia a fallos, Escalabilidad extrema con equipo mínimo

Funciones prefijas, infijas y postfijas en Haskell

	Función prefija	Función infija	Función postfija
¿Qué es?	La función se escribe antes de sus argumentos	La función se escribe entre argumentos	La función se escribe después de sus argumentos
Ejemplo de función	La función suma (+)	El operador suma (+)	El operador composición (.)
Ejemplo en Haskell	add :: Int -> Int -> Int add = (+)	3 + 5	(f . g) x
Resultado	add 3 5 -- 8	3 + 5 -- 8	(f . g) x - Aplica g a x y luego f
Características principales	- Es la forma más común en Haskell para llamar funciones - Es muy útil para funciones de varios parámetros	- Es la notación que usa operadores - Hace que las expresiones sean más naturales de leer	- Se usa principalmente para composición de funciones - Se lee de derecha a izquierda

Intervalos, operadores y evaluacion perezosa

1 En una introduccion a Haskell, se suele llamar intervalos o rangos a las secuencias que escriben mediate la notacion ... Esta notacion genera una lista de valores siguiendo una enumeracion, no representa un intervalo continuo de números reales, si no una secuencia de elementos. su

Su principal ventaja es que permite generar secuencias de manera sencilla sin tener que escribir cada elemento individualmente

Intervalos simples: generan una secuencia de numeros consecutivos
[1..10] Resultado [1,2,3,4,5,6,7,8,9,10]

Intervalos descendentes: Se pueden generar secuencias descendentes especificando el siguiente elemento [10,9..1] Resultado [10,9,8,7,6,5,4,3,2,1]

Intervalo de caracteres tambien genera secuencias alfabeticas
['a'..'z'] Resultado: todas las letras del alfabeto

2 Operadores: son simbolos o funciones que permiten realizar operaciones entre valores, en hoskes se utilizan para realizar calculo matematicos comparaciones logicas

Operadores aritmeticos: +, -, *, /, division entera (div) residuo (mod)

Operadores relacionales permiten comparar dos valores y devuelven un resultado booleano: ==, /=, >, <, >=, <=

Operadores logicos Se ulizan para combinar funciones.

&& AND (Y Logico
|| OR (O lógico)
not Negacion logica

3. Evaluacion perezosa

Es una de las caracteristicas más importantes de Haskell, consiste en que el lenguaje no calcula una expresión inmediatamente, sino que espera hasta que su resultado sea realmente necesario

Esto significa que Haskell puede trabajar con listas infinitas y realizar operaciones sin necesidad de calcular todos sus elementos desde el principio

Ventajas

Reduce el calculo innecesario
Permite manejar estructuras de datos infinitas
Facilita la creacion de programas modulares
Mejora el Rendimiento cuando no se necesitan todos los resultados

Desventajas

Puede dificultar la comprension del momento en que se realizan los calculos
Provoca un consumo elevado de memoria
dificulta la deteccion de algunos errores de rendimiento
Puede generar programas mas dificiles de depurar